

Rapport BE POO Bin Inventory

Valentin CZUBEK et Mathieu MUELAS

Introduction

Ceux qui bricolent se reconnaîtront ! Lorsqu'on bricole on commence vite à être désorganisé et les outils et composants s'éparpillent assez vite. Une première solution est de mettre en place une forme de rangement comme des casiers à composants électroniques, de ranger ses outils dans un tiroir un à endroit bien précis. Cependant ces solutions ont un désavantage : la flexibilité ! Si on ajoute des éléments il faut se souvenir de l'endroit où nous les avons rangés et cela est d'autant plus dur qu'on ne les utilise pas souvent (surtout pour les composants électroniques !).

Pour avoir une flexibilité physique nous avons trouvé un système de rangement totalement modulaire il est nommé [Gridfinity](#) ! C'est un système de cadrillage standardisé sur lequel on pose des boîtes "bin" de dimensions de multiples entiers de 42x42x7mm. Ce système est modifiable à souhait pour accommoder tous types d'outils de composants et bien plus encore ! Quelques exemples :



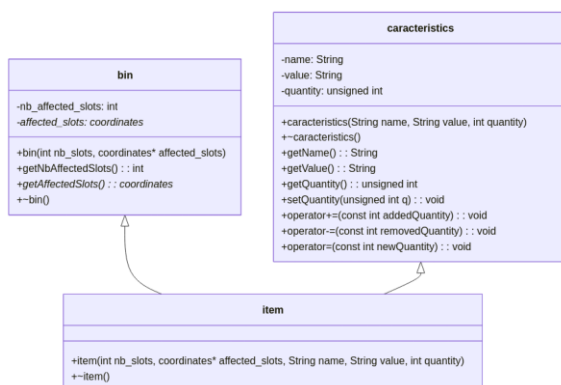
Source : www.reddit.com



Source : www.reddit.com

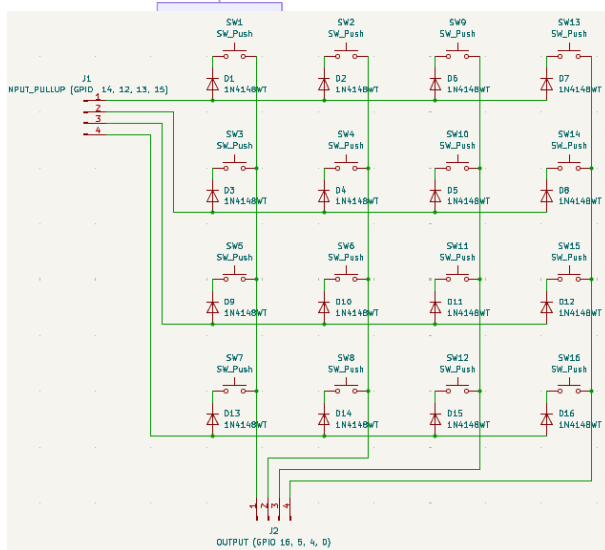
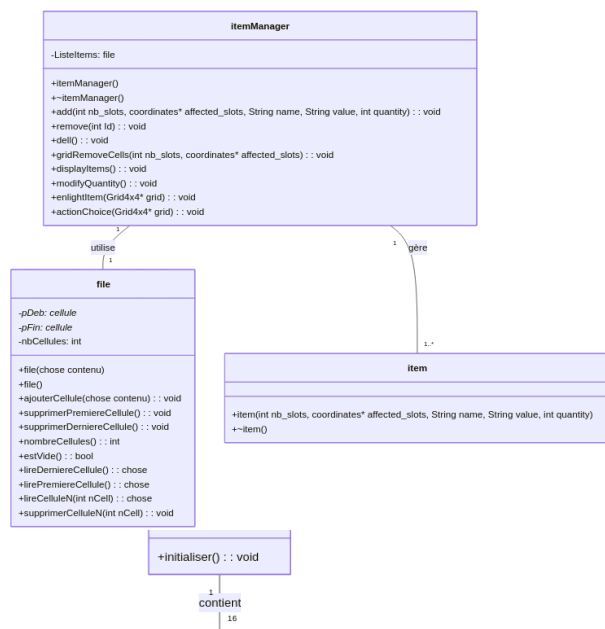
Le dernier problème était de retrouver ses composants. Pour cela l'idée de l'électroniser faisait sens : ajouter un capteur en dessous de chaque emplacement pour détecter l'ajout ou la suppression d'une "bin" et une LED qui permettra de retrouver physiquement la "bin" contenant ce que l'on recherche depuis l'interface utilisateur.

Fonctionnement



Les items sont stockés grâce à 3 classes : la classe item qui va hériter de deux classes : la classe bin qui comportera les slots affectés par l'item (il peut y en avoir plusieurs selon ce qu'on stocke : un grand outil, un petit bac de vis, de LEDs...)

On a une partie gestion des items avec l'item manager qui a pour mission de stocker les items qu'on ajoute dans la file (file qui ajoutera des cellules on en enlèvera). Il a également pour mission de réaliser l'affichage



de ce qui est présent dans la file et de sa modification grâce à l'interface utilisateur en connexion série.

Il y a une autre partie qui est plutôt la partie hardware. C'est la classe détection manager qui permet de détecter lorsqu'on pose une nouvelle bin, elle utilise alors la communication série pour nous demander ce que contient la bin. Elle permet également de supprimer de la file (en utilisant une fonction d'item manger) une bin lorsqu'on l'enlève de l'inventaire (détection d'un capteur passant de 0 à 1 ou de 1 à 0).

La récupération des capteurs se fait avec une fonction différente (lire boutons poussoirs) qui va venir allumer successivement chaque ligne pour récupérer les états de chaque colonne, on a ainsi besoin de 4+4 GPIO pour une matrice de 16 capteurs (4x4) au lieu d'utiliser 16 GPIO, ce qui n'aurait pas été possible car nous n'avons pas ce nombre de broches. Cette fonction enregistre les états dans la grille (grid). Voici le schéma électrique des capteurs (simples boutons poussoirs / contacts).

Remarque : on utilise des diodes sur chaque bouton pour éviter les mauvaises lectures si on appuie sur plusieurs boutons. Exemple : si on appuie sur SW1, SW3 et SW4 et qu'il n'y a pas de diode lorsqu'on va tester la 1ere ligne le courant va passer par SW1 puis

SW3 remonter dans la ligne 2 et remonter sur la colonne 2 grâce à SW4 on détectera donc un signal sur la colonne 2 et on va l'interpréter comme le fait que SW2 est allumé alors que ce n'est pas le cas.

Finalement dans notre cas nous nous sommes aperçus que la broche D8 ou GPIO15, ne fonctionne pas comme entrée (sûrement car ça doit être une broche spécialisée), nous n'en avons pas d'autre donc notre programme sur cette carte ne fonctionnait que pour un Grid4x3 mais le principe de fonctionnement lui a été prouvé.

Les LEDs sont des LEDs adressables (c'est une sorte de registre à décalage), nous utilisons une librairie pour cela. Elle permet de définir le nombre de LEDs, le type de bandeau, un tableau de couleur des LEDs et on peut les mettre à jour d'un coup. Nous avons donc un programme qui s'occupe de retranscrire les couleurs de la matrice (grid) dans le tableau et utilisons ensuite la commande pour les actualiser.

La fonction qui allume les LEDs est ensuite sensée être utilisée mais nous avons finalement fait légèrement différemment pour que le code fonctionne. (Il aurait fallu que la fonction qui change l'état des LEDs dans le tableau (pas en réel) soit non bloquante et qu'on re-rentre dedans après un temps donné pour les désactiver). Pour récapituler : la partie capteurs a bien une partie software et hardware mais la partie LEDs a une partie software dans laquelle nous avons dû ajouter des bouts de code hardware dedans...

Conclusion

Ce projet d'inventaire électronique est une idée innovante qui met en œuvre la programmation orientée objet en C++ tout en offrant un cas concret d'utilisation. Il a fonctionné : nous avons pu grâce à une interface simple afficher les items présents dans notre inventaire, puis en choisir pour le faire ressortir visuellement en réel grâce à la LED et ainsi retrouver notre composant facilement. Notre système est modulable : on peut poser des bin utilisant un arrangement de slots particulier (2x1, 1x1 et plus avec des formes bizarres si désiré).

Ce projet est encore améliorable ce qui le rend très pertinent et intéressant :

- Possibilité d'ajouter des fonctionnalités avec un lecteur RFID sous chaque slot ce qui permettrait d'identifier chaque bin ou le mettre séparément pour d'abord identifier la bin puis la poser sur les capteurs.
- Comme on utilise un esp8266 on pourrait héberger une petite interface web qui permettrait d'ajouter, enlever plus facilement des items, de mieux les filtrer, ajouter des "tag" (composant électronique, outil, visserie, module...)
- Faire en sorte que les données ne soient pas perdues lorsqu'on reset l'esp8266 (avec une base de données ou un fichier JSON, ou utilisation de mémoire flash...)